

Speed Car Care — System Documentation

1. System Description

1.1 Overview

Speed Car Care is a comprehensive, cloud-based business management system designed and developed for a car care service center located in Egypt. The center offers three core service categories — **car washing**, **oil changing**, and **carpet cleaning** — each operating from a different physical location within the same facility. The system automates the day-to-day operations of the business including invoice creation, service queue management, customer identification, loyalty point tracking, oil product inventory control, and customer feedback collection.

The application is built as a **single-page web application (SPA)** that runs entirely in the browser. It communicates with a backend consisting of **serverless API functions** deployed on Vercel, which in turn interact with a **PostgreSQL database** hosted on Supabase and a **transactional email service** provided by Brevo (formerly Sendinblue). The system is designed to be used by employees on desktop computers or tablets at the service counter, while the business owner interacts with the system remotely through email-based invoice approval.

1.2 Problem Statement

Before this system was developed, Speed Car Care relied on manual paper-based record keeping for invoices, customer history, and inventory tracking. This led to several operational challenges:

- **No centralized customer records:** There was no quick way to look up a returning customer's history, previous services, or vehicle information.

- **No queue visibility:** Employees and customers had no way to know how many people were ahead of them in the service queue or estimate wait times, especially for carpet cleaning which takes significantly longer (days vs. minutes).
- **No approval workflow:** Invoices were created and stored without any owner oversight, increasing the risk of errors or unauthorized entries.
- **No loyalty tracking:** The business had no mechanism to reward repeat customers or track how frequently they used each service type.
- **No inventory control:** Oil product stock levels were tracked informally, leading to stockouts and over-ordering.
- **No customer feedback channel:** There was no structured way for customers to provide feedback about their experience.

This system was built to solve all of these problems through a unified digital platform.

1.3 Core Features — Detailed Breakdown

1.3.1 Invoice Management System

The invoice system is the central feature of the application. When a customer arrives at the service center, the employee opens the system and follows this workflow:

1. **Customer Identification:** The employee enters the customer's mobile phone number. The system immediately queries the database to check if this customer has visited before. If a match is found, the customer's name, car registration number, and accumulated loyalty points are automatically populated into the form, saving time and reducing data entry errors.
2. **Service Selection:** The employee navigates between three service tabs — Washing (غسيل), Oil (زيت), and Carpet (سجاد). Each tab displays the available services with their prices. The employee selects the relevant services by clicking on them, and each selected service is added to the running invoice. The system maintains separate subtotals for each service

category (wash_total, oil_total, carpet_total) as well as a combined grand total.

3. **Invoice Staging:** When the employee presses "Save Invoice" (حفظ الفاتورة), the invoice is **not immediately committed** to the permanent database. Instead, it is inserted into a staging table called `pending_invoices`. This implements a **maker-checker workflow** where the employee (maker) creates the invoice, and the business owner (checker) must approve it before it becomes permanent.
4. **Email Approval:** Simultaneously, the system sends a richly formatted HTML email to the business owner via the Brevo email API. This email contains all the invoice details — customer name, phone number, car number, kilometer readings (for oil changes), a breakdown of all selected services with individual prices, carpet code (if applicable), and the total amount. The email also contains two prominent action buttons: "موافقة" (**Approve**) and "رفض" (**Reject**). Each button is a link to a serverless API endpoint with the pending invoice ID and the desired action encoded as URL parameters.
5. **Owner Decision:** When the owner clicks "Approve", the system fetches the pending record, moves the invoice data to the permanent `invoices` and `daily_invoices` tables, updates the customer's loyalty points, and then deletes the record from `pending_invoices` to prevent duplicate approvals. If the owner clicks "Reject", the pending record is simply deleted without saving anything. In both cases, the owner sees a confirmation page in their browser.

1.3.2 Multi-Queue System

Because Speed Car Care operates washing, oil, and carpet services from **different physical locations** within the facility, the system implements **three independent service queues**. When an invoice is saved, the system calculates the queue position for each service type separately:

- **Wash Queue:** Counts how many pending invoices with wash services were submitted before the current one. Each person ahead represents approximately **30 minutes** of wait time. The

alert shows the queue position and estimated wait in minutes or hours.

- **Oil Queue:** Same logic as wash — counts pending oil service invoices submitted before the current one, with a **30-minute** per-person estimate.
- **Carpet Queue:** Counts pending carpet service invoices. However, carpet cleaning takes significantly longer, so each person ahead represents approximately **2 days** of wait time. The alert displays the wait time in days.

The queue calculation uses the auto-increment database ID (`row.id < pendingId`) to determine ordering. This approach is resilient to concurrent submissions because the system inserts the invoice record first, then counts existing records, ensuring that simultaneous submissions from different terminals will always see each other's records.

1.3.3 Carpet Code Tracking System

Carpet cleaning orders require a unique tracking code because carpets are physically removed from the customer's vehicle and processed at a separate facility over multiple days. The carpet code serves as a receipt and tracking identifier.

Code Assignment Rules:

- Codes start at **100** and increment sequentially.
- Each customer (identified by phone number) gets **the same code** every time they order carpet cleaning. This means a returning customer always has the same carpet reference number.
- Different customers always get **different codes**. No two phone numbers can share the same code.

Concurrency Safety: To prevent race conditions where two simultaneous requests could generate the same carpet code, the system uses a **PostgreSQL stored function** (`assign_carpet_code`) that acquires an exclusive table lock before reading the maximum code and inserting a new one. This guarantees atomicity — even if 10 employees submit carpet invoices at the exact same millisecond, each will get a unique sequential code.

The carpet code is displayed in two places:

1. In the **browser alert** shown to the employee after saving the invoice.
2. In the **approval email** sent to the business owner.

The code is also stored inside the `invoice_data` JSON in `pending_invoices`, and when the invoice is approved, it carries through to the permanent `invoices` and `daily_invoices` tables.

1.3.4 Customer Loyalty Points System

The system tracks loyalty points per customer per service category. Each time a customer uses a service, they earn points:

- **Wash points:** 1 point per wash service selected
- **Oil points:** 1 point per oil service selected
- **Carpet points:** 1 point per carpet service selected

Points are stored in the `customer_points` table and are only awarded when an invoice is **approved** by the owner (not when it's first submitted). When a returning customer's phone number is entered, their accumulated points are displayed on screen, allowing the employee to offer rewards or discounts at their discretion.

1.3.5 Oil Product Inventory Management

The system includes a stock management module for oil products. It allows the employee to:

- **View all oil products** with their names, prices, and current stock levels (fetched from the `oil_products` table).
- **Update stock quantities** when new inventory arrives or after conducting stock checks.

This prevents the common problem of an employee promising an oil change to a customer only to discover the required oil product is out of stock.

1.3.6 Customer Feedback System

The system provides a feedback form that captures:

- **Customer name and phone number** (for follow-up)
- **Star rating** (numerical score)

- **Feedback type** (category of feedback)
- **Written message** (free-text comments)

When feedback is submitted, two things happen simultaneously:

1. The feedback is **saved to the database** in the `customer_feedback` table for long-term record keeping and analysis.
2. An **email alert** is sent to the business owner via Brevo, containing all the feedback details so the owner can respond promptly to customer concerns.

1.3.7 Email Notification System

The system uses the Brevo (Sendinblue) transactional email API for two types of notifications:

1. **Invoice Approval Emails:** Sent when an employee saves an invoice. Contains the full invoice details in a styled Arabic RTL HTML template with the business branding (dark theme, red accent color), plus approve/reject action buttons.
2. **Feedback Alert Emails:** Sent when a customer submits feedback. Contains the customer's name, phone, rating, feedback type, and message.

Both email types are sent from and to `nassaryehya26@gmail.com`, using Brevo as the SMTP relay to ensure reliable delivery.

1.4 Technology Stack

Layer	Technology	Justification
Frontend	HTML5, CSS3, Vanilla JavaScript	Single-file SPA (<code>index-final.html</code>) for simplicity and fast loading. No build tools or frameworks required. Arabic RTL layout with dark theme.

Backend	Vercel Serverless Functions (Node.js)	Zero infrastructure management. Each API endpoint is an independent function that scales automatically. Cold starts are minimal for the expected traffic volume.
Database	Supabase (PostgreSQL)	Managed PostgreSQL with built-in REST API, real-time subscriptions, and row-level security. The @supabase/supabase-js client library provides a convenient JavaScript interface. Supports stored functions (PL/pgSQL) for atomic operations.
Email	Brevo (Sendinblue) API	Reliable transactional email delivery with the @getbrevo/brevo SDK. Free tier supports the expected volume of invoice approval and feedback emails.
Hosting	Vercel	Automatic deployments from GitHub. Global CDN for static assets. Native support for serverless functions in the /api directory.
Version Control	GitHub	Source code repository connected to Vercel for continuous deployment.

1.5 Database Schema

pending_invoices

Column	Type	Description
id	SERIAL (PK)	Auto-increment ID, used for queue ordering
invoice_data	JSONB	Full invoice object (customer info, services, totals, carpet_code)

points_data	JSONB	Points to award on approval {wash, oil, carpet}
created_at	TIMESTAMPTZ	Automatic timestamp

invoices & daily_invoices

Column	Type	Description
customer_name	TEXT	Customer's full name
mobile_number	TEXT	Customer's phone number (used for lookup)
car_number	TEXT	Vehicle registration number
current_km	TEXT	Current odometer reading
next_km	TEXT	Next service odometer target
wash_total	NUMERIC	Subtotal for wash services
oil_total	NUMERIC	Subtotal for oil services
carpet_total	NUMERIC	Subtotal for carpet services
wash_services	TEXT	List of wash service names
oil_services	TEXT	List of oil service names

carpet_services	TEXT	List of carpet service names
total	NUMERIC	Grand total of all services
carpet_code	INT	Unique carpet tracking code (nullable)

customer_points

Column	Type	Description
customer_name	TEXT (PK)	Customer identifier
wash_points	INT	Accumulated wash service points
oil_points	INT	Accumulated oil service points
carpet_points	INT	Accumulated carpet service points

carpet_codes

Column	Type	Description
mobile_number	TEXT (PK)	Customer phone (unique constraint)
carpet_code	INT (UNIQUE)	Sequential code starting from 100

oil_products

Column	Type	Description
id	SERIAL (PK)	Product ID
name	TEXT	Product name
price	NUMERIC	Unit price
stock	INT	Current stock quantity

customer_feedback

Column	Type	Description
customer_name	TEXT	Customer name
customer_phone	TEXT	Customer phone
rating	INT	Star rating (1-5)
feedback_type	TEXT	Category of feedback
message	TEXT	Written feedback message

1.6 API Endpoints

Endpoint	Method	Description
/api/saveInvoice	POST	Inserts invoice to pending table, calculates 3 queue positions, assigns carpet code via DB function, sends approval email to owner
/api/processApproval	GET	Approves (moves to permanent tables + updates points) or rejects (deletes) a pending invoice
/api/approveInvoice	GET	Legacy approve-only endpoint
/api/getCustomer	POST	Looks up customer by phone, returns their info and loyalty points
/api/getProducts	GET	Returns all oil products with name, price, stock
/api/updateStock	POST	Updates stock quantity for a specific oil product
/api/sendFeedback	POST	Saves feedback to database and sends email alert to owner

1.7 Security & Access Control

- **Environment Variables:** All sensitive credentials (Supabase URL, Supabase service role key, Brevo API key) are stored as Vercel environment variables and are never exposed to the frontend.
- **Server-Side Only Access:** Database operations are performed exclusively through serverless functions. The frontend has no direct access to the database.
- **Supabase Service Role Key:** Used for server-side operations, bypassing row-level security for trusted backend operations.

- **No Authentication:** The current system does not implement user login — it is designed for use within a trusted internal network at the service center.

1.8 Concurrency Handling

The system addresses race conditions that occur when multiple employees submit invoices simultaneously:

1. **Insert-First Pattern:** The invoice is inserted into `pending_invoices` before any calculations are performed. This ensures that concurrent requests will see each other's records when querying for queue positions.
2. **ID-Based Queue Ordering:** Queue positions are calculated based on `row.id < pendingId` rather than `count - 1`, ensuring each concurrent request gets a unique, accurate position.
3. **Database-Level Locking for Carpet Codes:** The `assign_carpet_code()` PostgreSQL function uses `LOCK TABLE carpet_codes IN EXCLUSIVE MODE` to ensure that only one transaction can assign a new code at a time, making duplicate codes impossible.

1.9 Deployment Architecture

The system follows a **JAMstack** (JavaScript, APIs, Markup) architecture:

1. The developer pushes code to a **GitHub** repository.
2. **Vercel** automatically detects the push and triggers a deployment.
3. Static files (HTML, CSS, JS) are distributed to Vercel's **global CDN**.
4. Files in the `/api` directory are deployed as **serverless functions** on AWS Lambda (via Vercel).
5. At runtime, the serverless functions communicate with **Supabase** (hosted PostgreSQL) and **Brevo** (email API) over HTTPS.